Inteligencia Artificial

ENRIQUE PELÁEZ Ph.D.

Escuela Superior Politécnica del Litoral – ESPOL

Facultad de Ingeniería en Electricidad y Computación – FIEC

Clase # 23 - PAO 2026 – I

Agenda

1. Búsqueda heurística (o búsqueda informada)
   1.1 Hill Climbing
   1.2 Simulated Annealing
   1.3 Best – First Search
   1.4 Fronteras de la búsqueda
   1.5 Ejemplo

Búsqueda heurística

- Búsqueda heurística (o búsqueda informada)
  - Del griego "heuriskein", que significa descubrimiento o hallazgo.
  - La heurística ayuda o proporciona información para guiar el proceso de búsqueda.
  - La búsqueda basada en heurística no siempre garantiza encontrar el resultado óptimo (la mejor solución de todas), pero obtendremos muy buenos resultados en un tiempo razonable.
  - La heurística se usa para resolver problemas complejos donde generalmente tenemos una explosión combinatoria, donde los algoritmos de búsqueda exhaustivos tienen costos inaceptables.

Búsqueda heurística

- Pero, la heurística es una técnica que **siempre** produce resultados, aunque a veces no son los mejores.
- Mucho de lo que hacemos en la vida real implica heurística; cuando buscamos soluciones o tomamos decisiones para problemas diarios.
- Por lo general, las soluciones funcionan … o funcionan lo suficientemente bien; y, cuando no funcionan, tratamos el problema de una manera diferente.
- La heurística no solo se aplica a los casos en que una solución óptima podría no existir, sino también a los casos en los que queremos encontrar una mejor solución.

Hill Climbing

- ***Hill Climbing***
  - Es una versión mejorada del algoritmo de generación y prueba, pero con heurística.
  - Desde el procedimiento de prueba, hay una retroalimentación para ayudar al generador de soluciones decidir, en qué dirección ir a continuación o si es necesario cambiar de dirección de búsqueda en el espacio de soluciones.
  - En este proceso, la búsqueda finaliza si no hay un próximo estado o un nodo razonable para moverse.

Simple Hill Climbing

1. Evaluar el estado inicial.
2. Repetir, hasta que se encuentre una solución, o ya no haya nuevos operadores que aplicar:

   - Seleccionar un nodo candidato y aplicar la función heurística

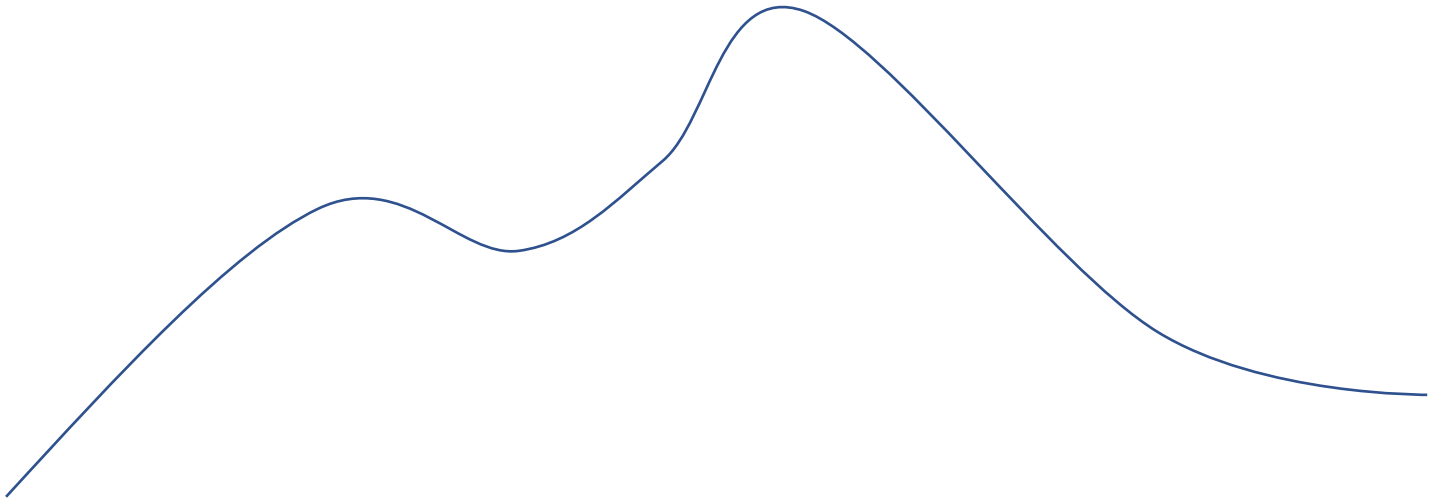
   - Evaluar el nuevo estado:

     - nuevo estado = objetivo ? $\rightarrow$ terminar
     - nuevo estado es mejor que el estado actual ? $\rightarrow$ avanzar con el nuevo estado actual

Hill Climbing: Disadvantages

Maximo local

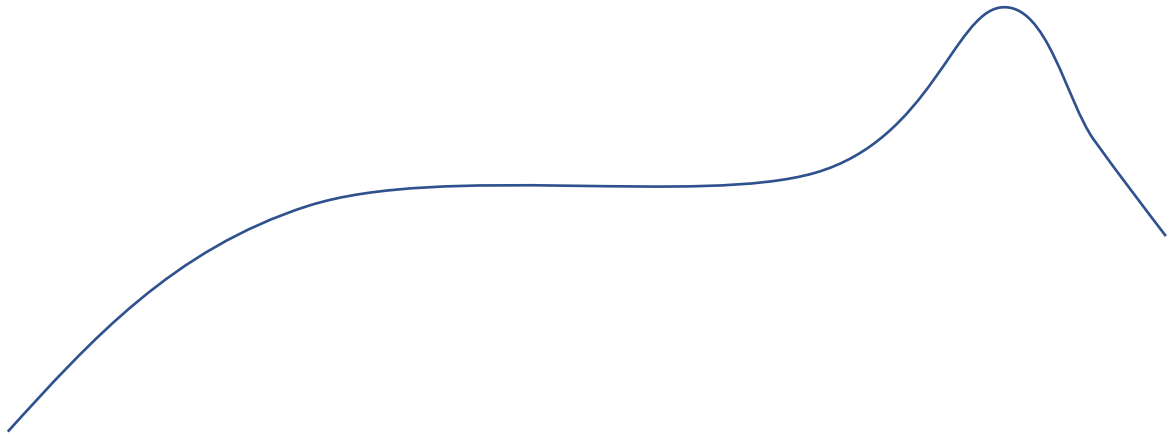
- Es el mejor estado o nodo en comparación con sus vecinos, pero no es el mejor entre otros, que podría estar adelante en el espacio de la solución.



Hill Climbing: Desventajas

Meseta

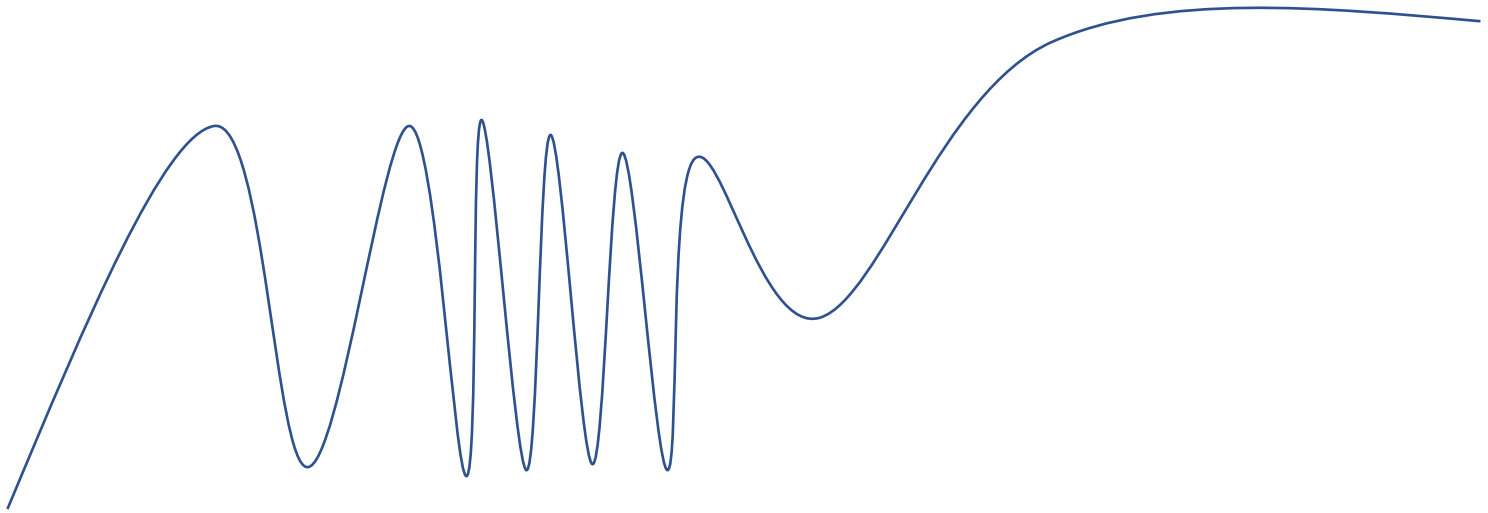
- Es un área plana del espacio de búsqueda en la que todos los estados vecinos tienen el mismo, o similar valor.



Hill Climbing: Desventajas

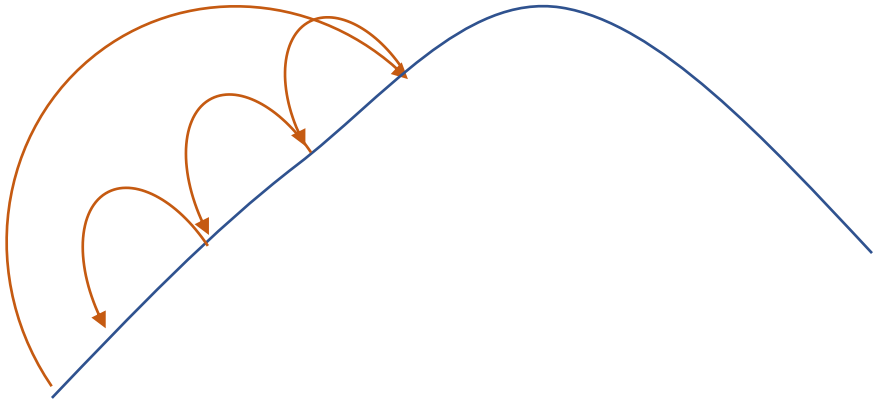
Acantilado

- Es un tipo especial de máximo local, que es imposible de cruzar con movimientos simples.



Simulated Annealing

- El recocido simulado es una versión mejorada del algoritmo Hill Climbing.
- Al principio, este algoritmo explora una buena parte del espacio de la solución, de tal manera que la solución final podría no verse afectada por el estado inicial.
- Como consecuencia, la posibilidad de quedar atrapado en un máximo local, meseta o acantilado, es menor.



Simulated Annealing

1. Evaluar el estado inicial.
2. Repetir hasta que se encuentre una solución o no haya nuevos operadores para aplicar:

   - Establecer T como un estado inicial

   - Selecciona y aplica un nuevo operador

   - Evaluar el nuevo estado:

     goal $\rightarrow$ terminar

     ΔE = Valor (estado actual) - Valor (nuevo estado)
     $\Delta E < 0 \rightarrow$ nuevo estado actual
     else $\rightarrow$ nuevo estado actual con probabilidad $e^{-\Delta E/kT}$

Best-First Search

- Intuiciones - hasta ahora hemos visto:

  - Búsqueda en amplitud primero: no quedar atrapado en caminos sin salida.
  - Búsqueda en profundidad primero: no todas las ramas tienen que expandirse.
  - Se podría combinar los dos, de manera que se siga un camino a la vez, pero cambiar de ruta cada vez que aparezca una ruta más prometedora que la actual.

Best-First Search

- **Best - First**
  1. **Lista open**: contiene los nodos generados.

     A los nodos se les ha aplicado la función heurística; pero aún no han sido probados; es decir, sus sucesores aún no se han generado. Esta cola puede considerarse como una cola de prioridad, en la que los elementos con mayor prioridad son los que tienen valores más prometedores, asignados por la función heurística.
  2. **Lista close**: contiene los nodos que se han examinado y no son el objetivo esperado.
  3. **Función heurística**: permite que el algoritmo busque primero las rutas más prometedoras.

Best-First Search

Función heurística:

$$f(n) = g(n) + h(n)$$

- g(n) = costo de la mejor ruta desde el estado inicial al nodo n.
- h(n) = costo de la mejor ruta desde el nodo n al estado objetivo.

Best-First Search

- Algoritmo Admisible

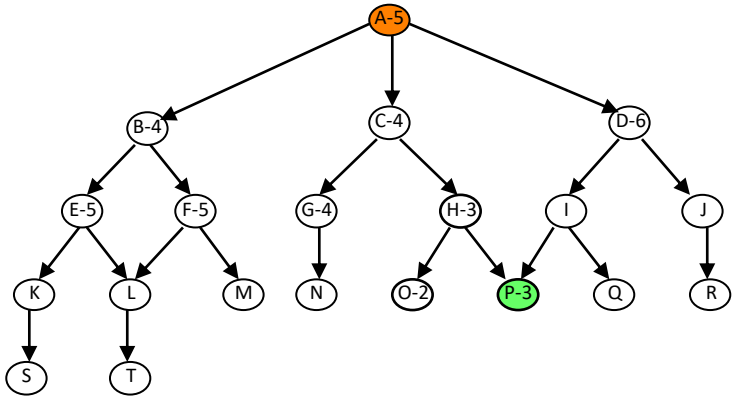
$$f(n) = g(n) + h(n)$$

Best-First Search

```
function best_first_search;

begin
   open := [Start];                                              % initialize
   closed := [ ];
   while open ≠ [ ] do                                           % states remain
      begin
         remove the leftmost state from open, call it X;
         if X = goal then return the path from Start to X
         else begin
                  generate children of X;
                  for each child of X do
                  case
                        the child is not on open or closed:
                           begin
                                 assign the child a heuristic value;
                                 add the child to open
                           end;
                        the child is already on open:
                           if the child was reached by a shorter path
                           then give the state on open the shorter path
                        the child is already on closed:
                           if the child was reached by a shorter path then
                              begin
                                 remove the state from closed;
                                 add the child to open
                              end;
                  end;                                           % case
                  put X on closed;
                  re-order states on open by heuristic merit (best leftmost)
               end;
return FAIL                                                      % open is empty
end.
```

Best-First Search

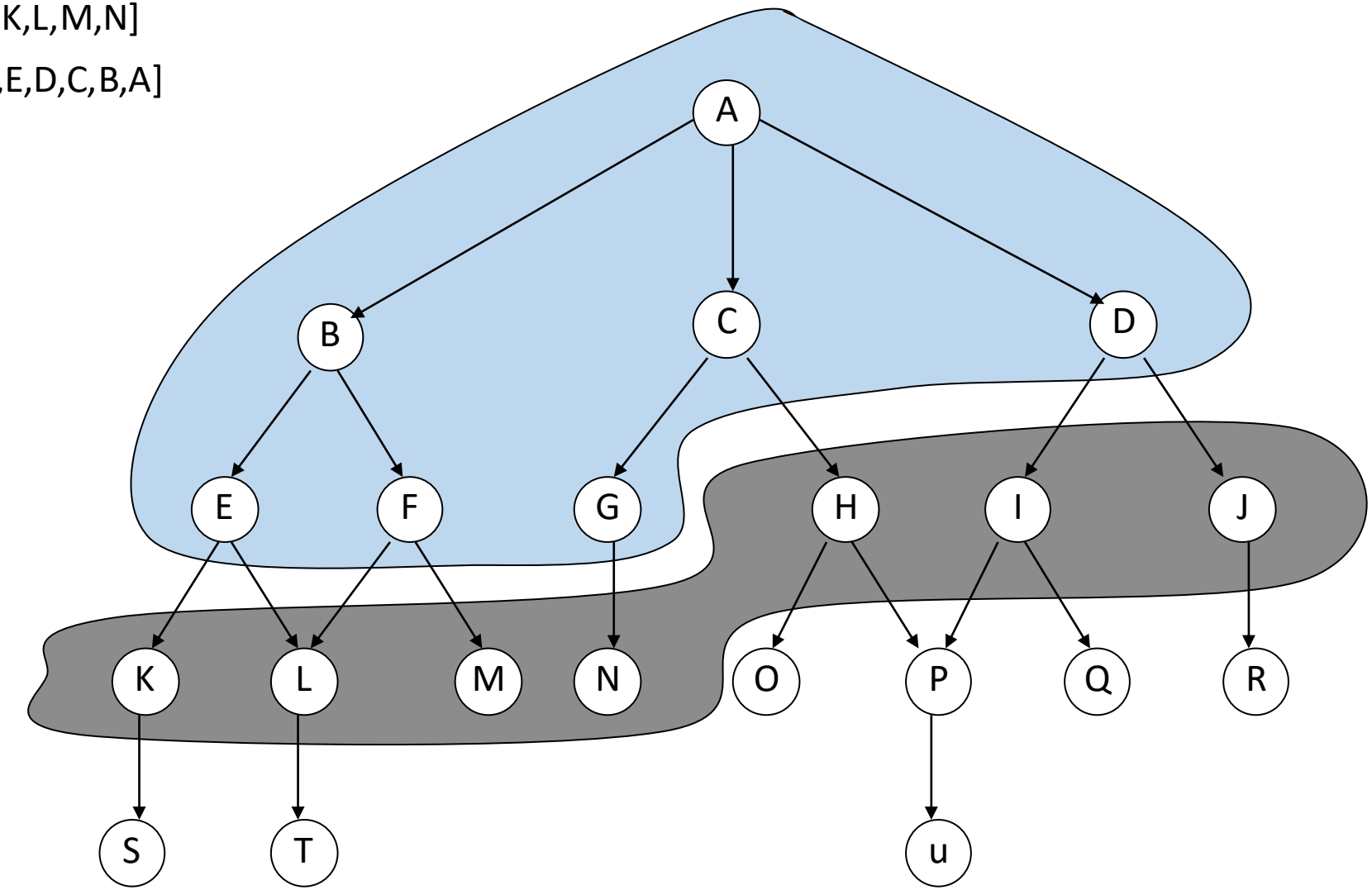


1. open = [A5]; close [ ]
2. open = [B4, C4, D6]; close [A5]
3. open = [C4, E5, F5, D6]; close [B4, A5]
4. open = [H3, G4, E5, F5, D6]; close [C4, B4, A5]
5. open = [O2, P3, G4, E5, F5, D6]; close [H3, C4, B4, A5]
6. open = [P3, G4, E5, F5, D6]; close [O2, H3, C4, B4, A5]
7. P3 Solution

Breadth - First

open = [H,I,J,K,L,M,N]

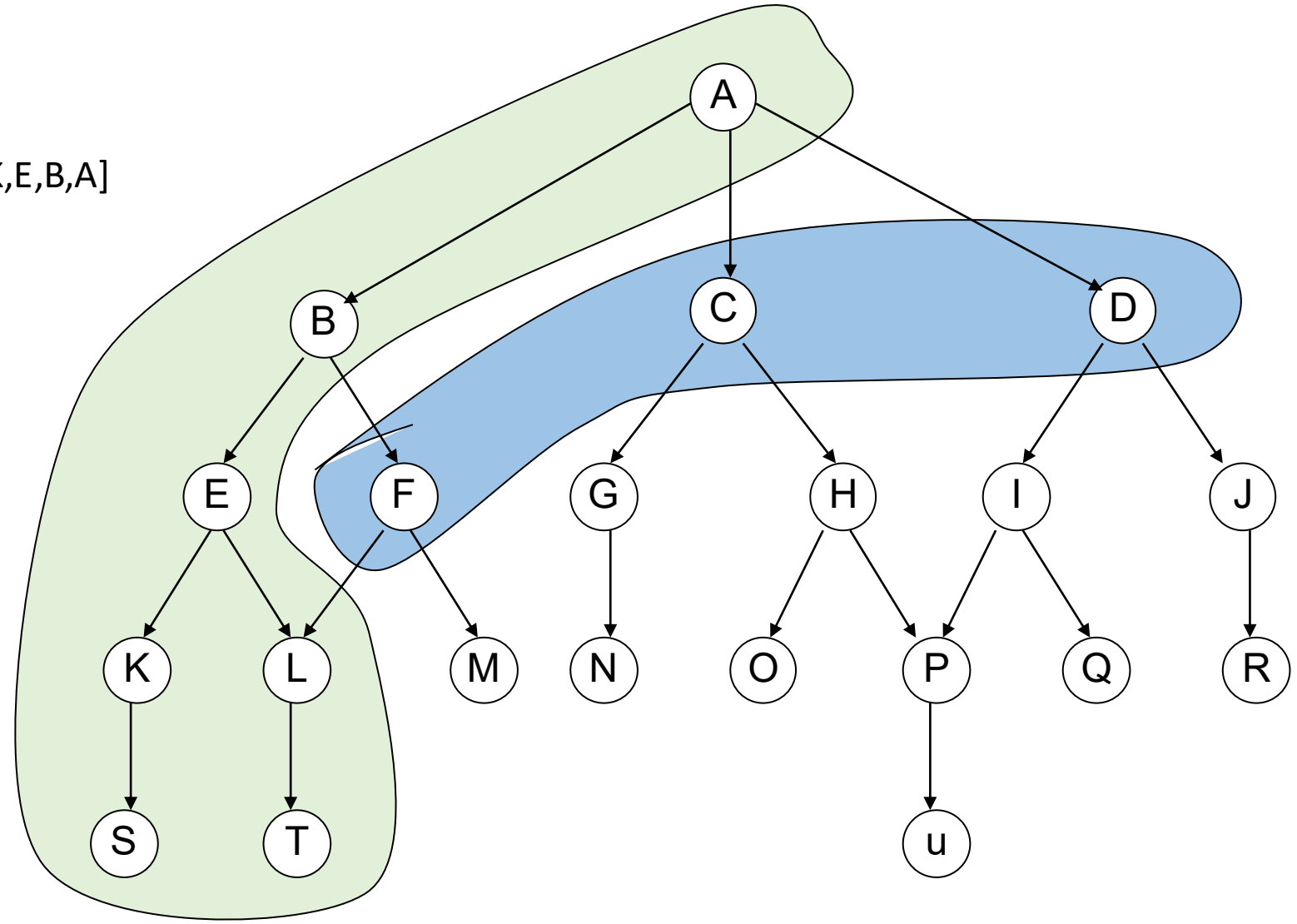
closed = [G,F,E,D,C,B,A]



Depth - First

open = [F,C,D]

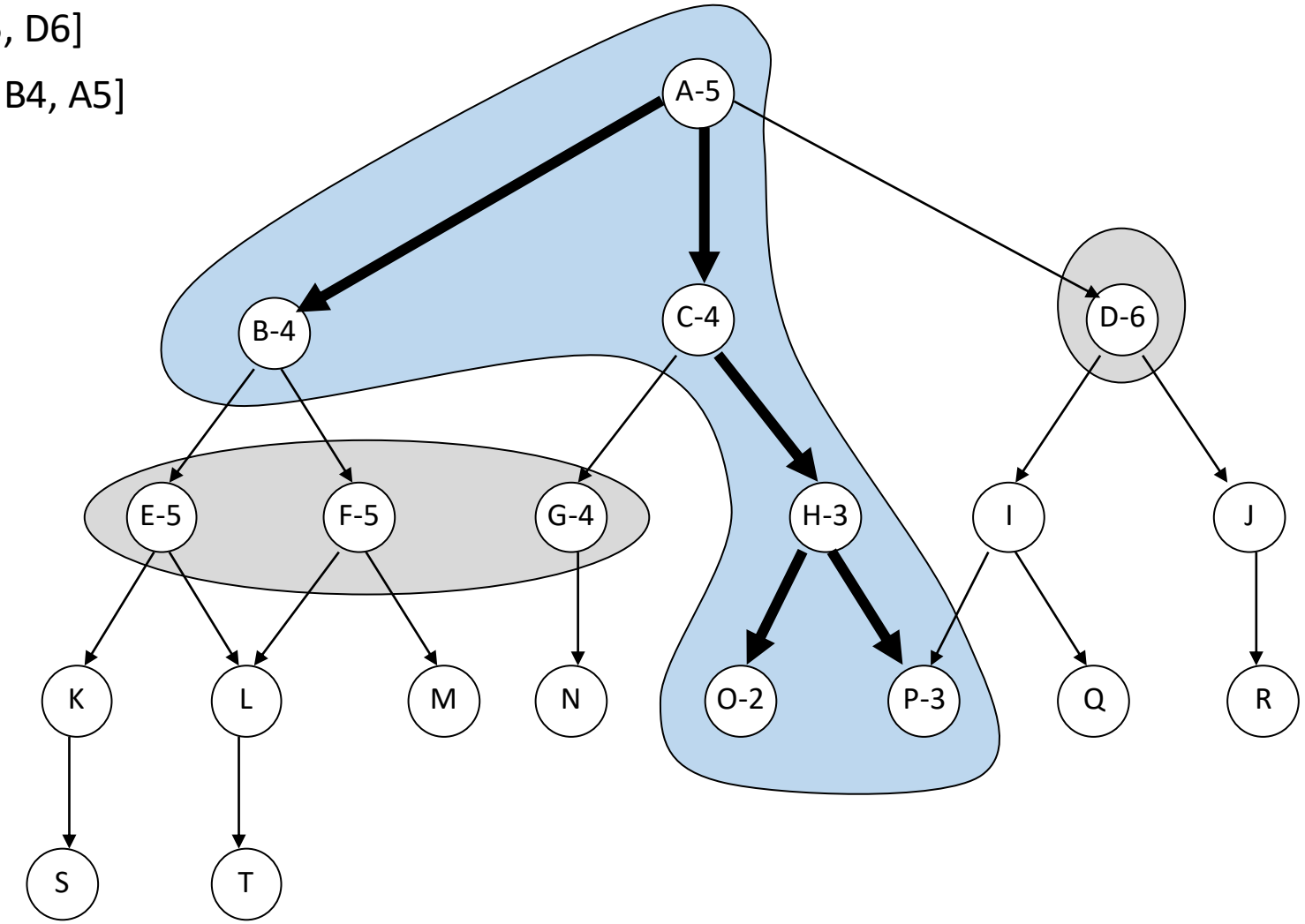
closed = [T,L,S,K,E,B,A]



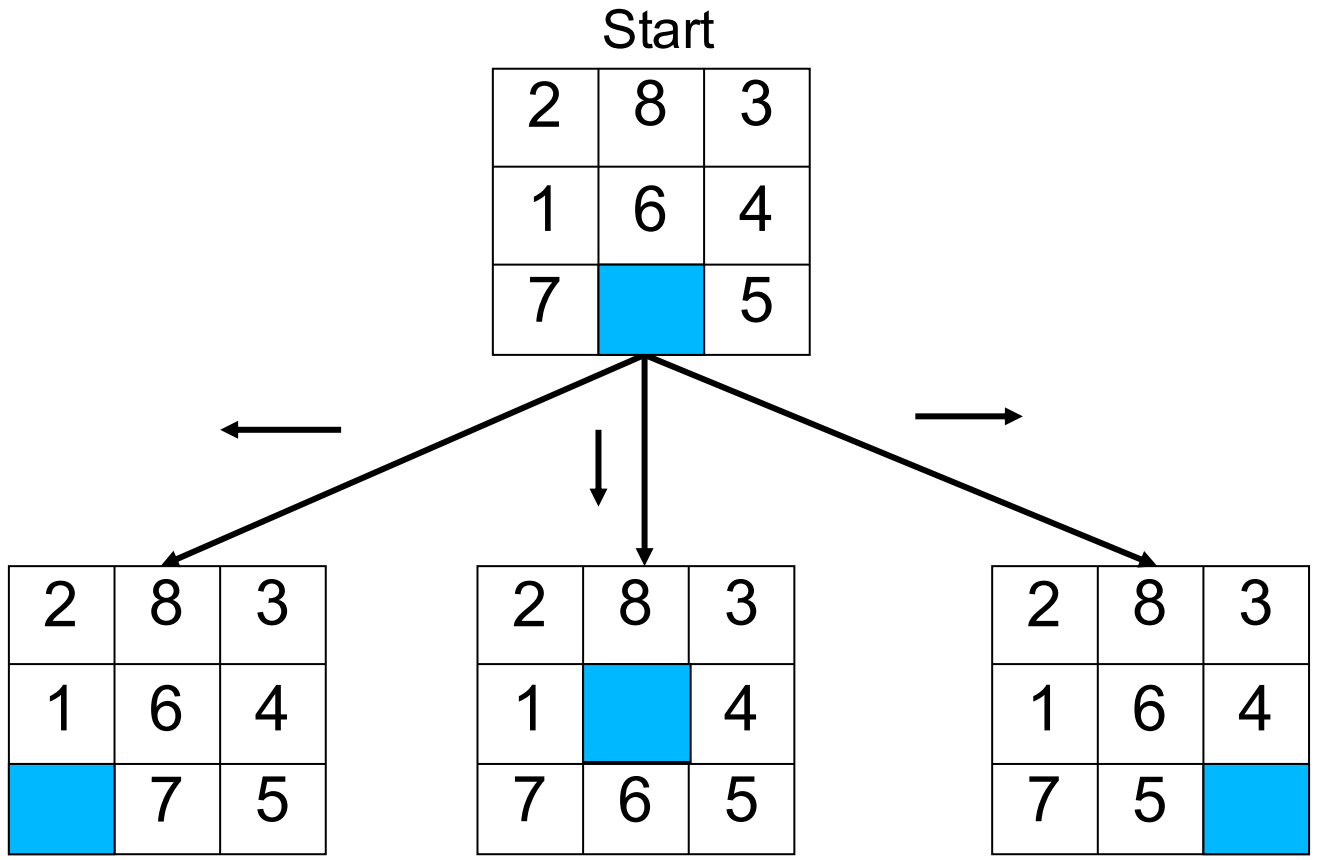
Best - First

open = [G4, E5, F5, D6]

close [O2, H3, C4, B4, A5]



Búsqueda heurística: Ejemplo



Goal

1	2	3
8		4
7	6	5

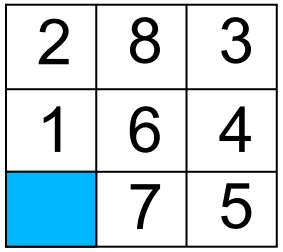
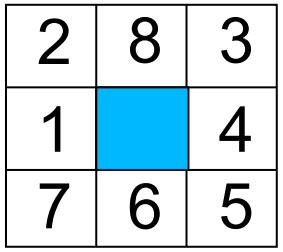
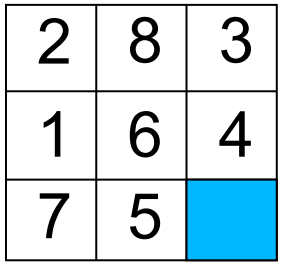
Búsqueda heurística: Ejemplo

Start

2	8	3
1	6	4
7		5

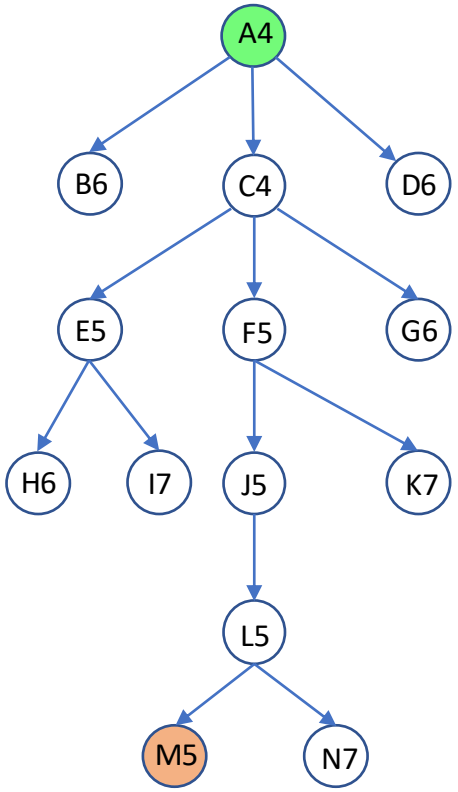
Goal

1	2	3
8		4
7	6	5

	5	6	
	3	4	
	5	6	
h(n)	# de fichas en posición incorrecta	Suma de distancias de fichas incorrectas	2 x # fichas reversas

Búsqueda heurística: Ejemplo

1. open = [a4]
   closed = [ ]
2. X = a4; open = [c4, b6, d6]
   closed = [a4]
3. X = c4; open = [e5, f5, b6, d6, g6]
   closed = [a4, c4]
4. X = e5; open = [f5, h6, b6, d6, g6, i7]
   closed = [a4, c4, e5]
5. X = f5; open = [j5, h6, b6, d6, g6, k7, i7]
   closed = [a4, c4, e5, f5]
6. X = j5; open = [l5, h6, b6, d6, g6, k7, i7]
   closed = [a4, c4, e5, f5, j5]
7. X = l5; open = [m5, h6, b6, d6, g6, n7, k7, i7]
   closed = [a4, c4, e5, f5, j5, l5]
8. X = m5; Objective m = Goal



Búsqueda heurística: Ejemplo

1. open = [a5]
   closed = [ ]
2. X = a5; open = [c5, b7, d7]
   closed = [a5]
3. X = c5; open = [f5, b7, d7, e7, g7]
   closed = [a5, c5]
4. X = f5; open = [j5, b7, d7, e7, g7, k7]
   closed = [a5, c5, f5]
5. X = j5; open = [l5, b7, d7, e7, g7, k7]
   closed = [a5, c5, f5, j5]
6. X = l5; open = [m5, b7, d7, e7, g7, k7, n7]
   closed = [a5, c5, f5, j5, l5]
7. X = m5; Objective m = Goal

